13 优化多线程应用

现代 CPU 每年都在增加越来越多的核心。截至 2020 年，你可以购买到一款 x86 服务器处理器，其核心数量超过 50 个！而拥有 8 个执行线程的中档台式机也是相当常见的配置。由于每个 CPU 中有如此强大的处理能力，如何高效利用所有的硬件线程成为了挑战。为了确保应用的未来成功，准备软件以便与不断增长的 CPU 核心数量良好地扩展非常重要。

多线程应用具有其自身的特点。在处理多个线程时，单线程执行的某些假设会失效。例如，我们不能再通过查看单个线程来识别热点，因为每个线程可能都有自己的热点。在流行的生产者-消费者[218]设计中，生产者线程可能大部分时间都在休眠。对这样一个线程进行分析不会揭示出我们的多线程应用为何扩展性不佳的原因。

13.1 性能扩展和开销

当处理单线程应用程序时，优化程序的某个部分通常会对性能产生积极的影响。然而，对于多线程应用程序来说，情况并非总是如此。有些应用程序中，线程 `A` 执行一个长时间运行的操作，而线程 `B` 则早早地完成了其任务，只是等待线程 `A` 完成。无论我们如何改进线程 `B`，应用程序的延迟都不会减少，因为它将受到运行时间较长的线程 `A` 的限制[219]。

这种影响被广泛称为安达尔定律[220]，它规定了并行程序的加速度受其串行部分的限制。图 75 描绘了理论上的加速度极限，作为处理器数量的函数。对于一个其中有 75% 是并行的程序，加速度因子将收敛到 4。

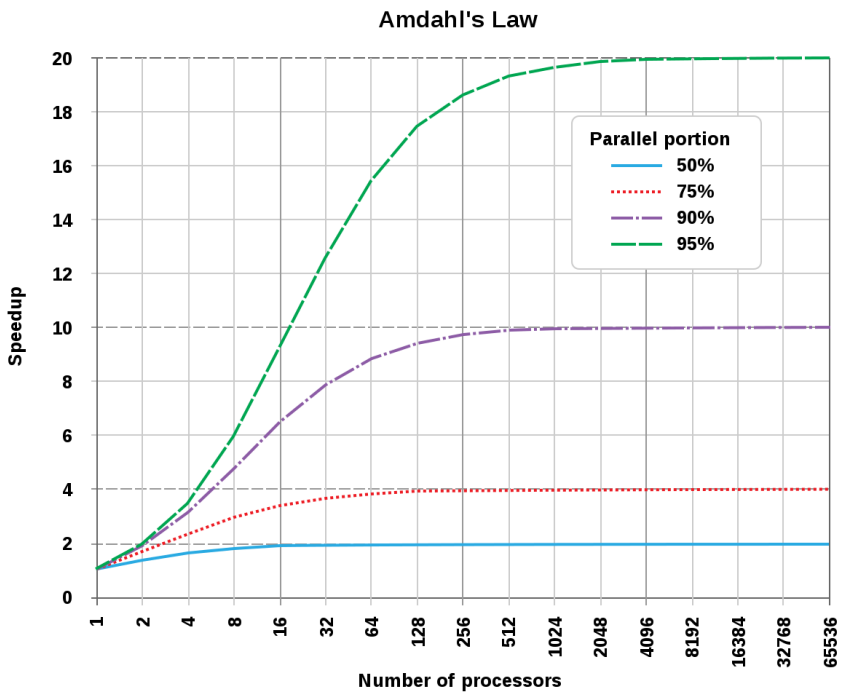


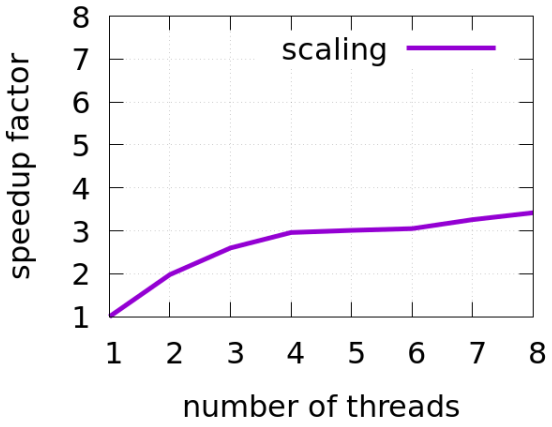
Figure 75: 根据安达尔定律，程序执行的理论加速度随处理器数量的变化。© Image by Daniels220 via Wikipedia.

[218] Producer-consumer pattern - https://en.wikipedia.org/wiki/Producer-consumer_problem

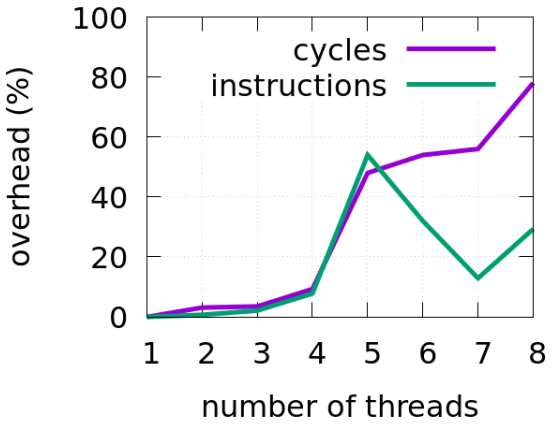
[219] 这不一定总是成立。例如，资源在线程/核心之间共享（如缓存）可能限制扩展性。此外，计算密集型基准测试往往只能在物理（而不是逻辑）核心数量上进行扩展，因为两个相邻的硬件线程共享同一个执行引擎。

[220] Amdahl's law - https://en.wikipedia.org/wiki/Amdahl's_law.

图 76a 显示了来自Starbench 并行基准套件的 `h264dec` 基准测试的性能扩展情况。我在拥有 4 个核心/8 个线程的 Intel Core i5-8259U 上进行了测试。请注意，在使用了 4 个线程之后，性能并没有显著提高。很可能，获取一个拥有更多核心的 CPU 不会提高性能。[221]



(a) 使用不同线程数量的性能扩展。



(b) 使用不同线程数量的开销。

Figure 76: 在 Intel Core i5-8259U 上的 h264dec 基准测试的性能扩展和开销。

事实上，进一步增加计算节点可能会导致逆向加速。这种效应由 Neil Gunther 解释为通用可扩展性法则 (Universal Scalability Law)[222]（USL），它是安达尔定律的一个扩展。USL 描述了计算节点（线程）之间的通信作为性能的另一个限制因素。随着系统的扩展，开销开始阻碍性能的增长。超过临界点后，系统的能力开始下降（见图 77）。USL 被广泛用于对系统的容量和可扩展性建模。

由 USL 描述的减速是由多种因素驱动的。首先，随着计算节点数量的增加，它们开始竞争资源（争用）。这导致额外的时间用于同步这些访问。另一个问题是资源在许多工作线程之间共享。我们需要在许多工作线程之间保持共享资源的一致状态（一致性）。例如，当多个工作线程频繁地更改全局可见对象时，这些更改需要广播到使用该对象的所有节点。突然之间，由于额外的一致性维护需求，通常的操作开始花费更多的时间来完成。在 Intel Core i5-8259U 上，h264dec 基准测试的通信开销可以在图 76b 中观察到。请注意，随着我们为任务分配超过 4 个线程，图表表明开销以经过的核心周期数的形式增加。[223]

优化多线程应用程序不仅涉及到本书迄今描述的所有技术，还涉及到检测和减轻竞争和一致性的前述影响。下一小

[221] 然而，它会受益于频率更高的 CPU。
[222] 通用可扩展性法则 - http://www.perfdynamics.com/Manifesto/USLscalability.html#tth_sEc1。
[223] 使用 5 和 6 个工作线程时，已完成的指令数量出现了一个有趣的峰值。这应该通过对工作负载进行分析来进行调查。

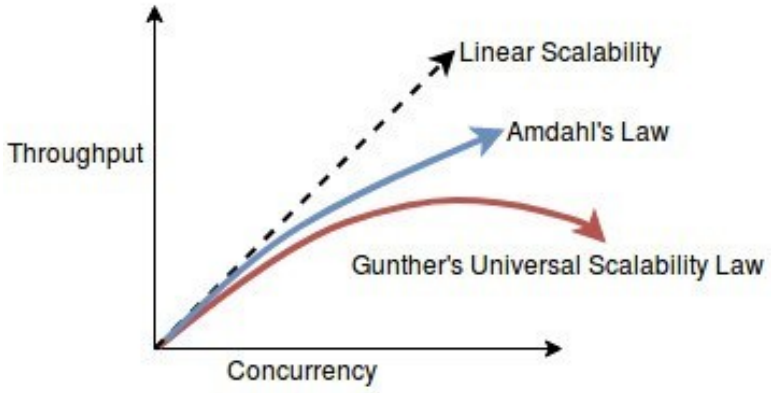


Figure 77: 通用可扩展性法则和安达尔定律。© Image by Neha Bhardwaj via Knoldus Blogs.

节将描述针对调优多线程程序的这些额外挑战的技术。

13.2 并行效率指标

在处理多线程应用程序时，工程师们应该谨慎分析诸如 CPU 利用率和 IPC（见 Chapter 4）等基本指标。某个线程可能表现出高 CPU 利用率和高 IPC，但实际上可能只是在一个锁上旋转。这就是为什么在评估应用程序的并行效率时，建议使用有效 CPU 利用率，该指标仅基于有效时间。[224]

13.2.1 有效 CPU 利用率

该指标表示应用程序有效地利用了可用的 CPU。它显示了系统上所有逻辑 CPU 的平均 CPU 利用率百分比。CPU 利用率指标仅基于有效时间，不包括并行运行时系统[225]和自旋时间引入的开销。100% 的 CPU 利用率意味着您的应用程序在运行期间始终使所有逻辑 CPU 内核保持忙碌 [Intel, 2023a]。

对于指定的时间间隔 T，可以计算有效 CPU 利用率如下：

$$\text{Effective CPU Utilization} = \frac{\sum_{i=1}^{\text{ThreadsCount}} \text{Effective Cpu Time(T,i)}}{\text{T} \times \text{ThreadsCount}}$$

13.2.2 线程数

应用程序通常具有可配置的线程数，这使它们能够在具有不同核心数的平台上有效运行。显然，使用比系统上可用的线程数少的线程来运行应用程序会浪费其资源。另一方面，运行过多的线程可能会导致较高的 CPU 时间，因为其中一些线程可能正在等待其他线程完成，或者时间可能被用于上下文切换。

除了实际的工作线程外，多线程应用程序通常还具有辅助线程：主线程、输入和输出线程等。如果这些线程消耗了大量时间，它们就需要专用的硬件线程。这就是为什么了解总线程数并正确配置工作线程数很重要。

为了避免线程创建和销毁的惩罚，工程师通常会分配一个线程池[226]，其中有多个线程等待由监督程序分配的任务以供并发执行。这对执行短暂任务特别有益。

[224] 诸如 Intel VTune Profiler 等性能分析工具可以区分在线程旋转时采集的分析样本。它们通过为每个样本提供调用堆栈来完成这项工作（见 Section 5.5.3）。

[225] 类似 `pthread`、`OpenMP` 和 `Intel TBB` 等线程库和 API 具有它们自己的线程创建和管理开销。

[226] 线程池 - https://en.wikipedia.org/wiki/Thread_pool。

13.2.3 等待时间

等待时间发生在软件线程由于阻塞或导致同步的 API 而等待时。等待时间是每个线程的；因此，总等待时间可能超过应用程序经过的时间 [Intel, 2023a]。

线程可以由于同步或抢占而被操作系统调度程序从执行中切换掉。因此，等待时间可以进一步分为同步等待时间和抢占等待时间。大量的同步等待时间很可能表明应用程序具有高度竞争的同步对象。我们将在接下来的章节中探讨如何找到它们。显着的抢占等待时间可以表明线程过度订阅问题 (oversubscription)[227]，这可能是由于大量的应用程序线程或与操作系统线程或系统上其他应用程序的冲突引起的。在这种情况下，开发人员应考虑减少线程总数或增加每个工作线程的任务粒度。

13.2.4 自旋时间

自旋时间是 CPU 忙于等待时间。当软件线程等待时，这种情况经常发生，因为同步 API 导致 CPU 轮询 [Intel, 2023a]。实际上，内核同步原语的实现更倾向于在一段时间内锁定旋转，而不是立即进行线程上下文切换（这是昂贵的）。然而，过多的自旋时间可能反映了无法进行有效工作的机会的丧失。

13.3 使用 Intel VTune Profiler 进行分析

Intel VTune Profiler 有一种专门针对多线程应用程序的分析类型，称为线程分析。其摘要窗口（见图 78）显示了有关整个应用程序执行的统计信息，识别了我们在 Section 13.2 中描述的所有指标。从有效 CPU 利用率直方图中，我们可以了解到有关捕获的应用程序行为的几个有趣事实。首先，平均而言，同时仅利用了 5 个硬件线程（图表中的逻辑核心）。其次，所有 8 个硬件线程同时活跃的情况非常罕见。

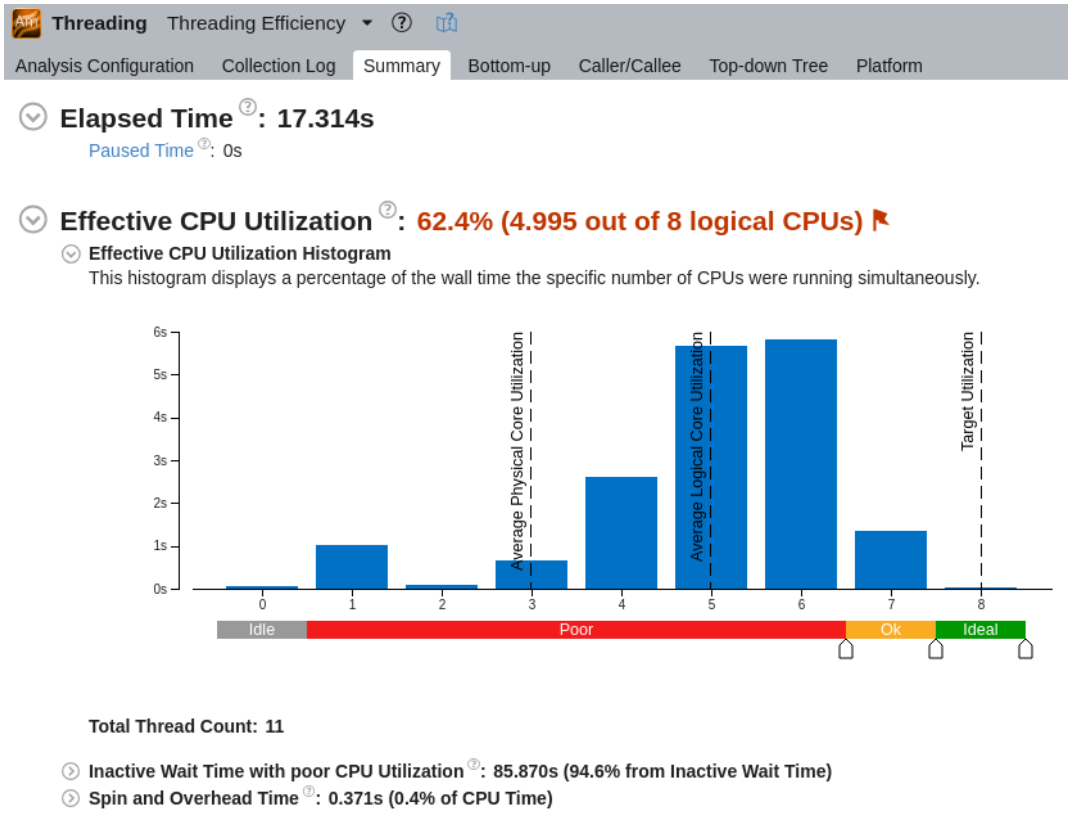


Figure 78: 来自Phoronix 测试套件中 x264 基准测试的 Intel VTune Profiler 线程分析摘要。

[227] 线程过度订阅 - https://software.intel.com/en-us/vtune-help-thread-oversubscription。

13.3.1 查找开销大的锁

接下来，工作流程建议我们识别最有争议的同步对象。图 79 显示了这些对象的列表。我们可以看到 `__pthread_cond_wait` 显然突出，但由于程序中可能有几十个条件变量，我们需要知道哪一个是导致 CPU 利用率不佳的原因。

Inactive Wait Time with poor CPU Utilization: **85.870s (94.6% from Inactive Wait Time)**

Inactive Sync Wait Time: 85.508s

Preemption Wait Time: 0.361s

Top functions by Inactive Wait Time with Poor CPU Utilization.

This section lists the functions sorted by the time spent waiting on synchronization or thread preemption with poor CPU Utilization.

Function	Module	Inactive Wait Time	Inactive Sync Wait Time	Inactive Sync Wait Count
__pthread_cond_wait	libpthread-2.27.so	84.596s	84.594s	24,903
__GI___pthread_mutex_lock	libpthread-2.27.so	0.889s	0.889s	79
[vmlinux]	vmlinux	0.374s	0.016s	453
[vtsspp]	vtsspp	0.005s	0.005s	126
__GI___pthread_mutex_unlock	libpthread-2.27.so	0.002s	0s	0
[Others]		0.004s	0.003s	58

Figure 79: 显示了来自 Phoronix 测试套件中 x264 基准测试的 Intel VTune Profiler 线程分析，显示了最有争议的同步对象。

要找出原因，我们可以简单地点击 `__pthread_cond_wait`，这将带我们到底部向上视图，如图 80 所示。我们可以看到导致线程等待条件变量的最频繁路径（等待时间的 47%）：`__pthread_cond_wait <- x264_8_frame_cond_wait <- mb_analyse_init`。

Threading | Threading Efficiency

Analysis Configuration | Collection Log | Summary | Bottom-up | Caller/Callee | Top-down Tree | Platform

Grouping: Function / Call Stack

Function / Call Stack	CPU Time	Inactive Wait Time		Inactive Wait Count		Module
		Inactive Sync Wait Time	Preemption Wait Time	Inactive Sync Wait Count	Preemption Wait Count	
__pthread_cond_wait	0.241s	89.428s	0.002s	26,375	4	libpthread-2.27.so
↖ x264_8_frame_cond_wait ← mb_analyse_init ←	0.216s	42.040s	0.002s	24,239	3	x264
↖ threadpool_thread_internal ← x264_stack_align	0.015s	29.530s	0.000s	1,464	1	x264
↖ x264_8_threadpool_wait ← encoder_frame_end	0.008s	14.377s	0s	491	0	x264
↖ lookahead_thread_internal ← x264_stack_align	0.002s	2.562s	0s	139	0	x264
↖ x264_8_lookahead_get_frames ← x264_8_encoc	0s	0.918s	0s	42	0	x264
__GI___pthread_mutex_lock	0.016s	0.895s	0.000s	87	1	libpthread-2.27.so
[vmlinux]	0.520s	0.017s	0.391s	467	1,134	vmlinux
[vtsspp]	0.002s	0.005s	0s	131	0	vtsspp
__pthread_cond_broadcast	0.084s	0.001s	0.001s	4	5	libpthread-2.27.so

Figure 80: 显示了来自 Phoronix 测试套件中 x264 基准测试的 Intel VTune Profiler 线程分析，显示了最有争议的条件变量的调用堆栈。

接下来，我们可以通过双击分析中相应行来跳转到 `x264_8_frame_cond_wait` 函数的源代码视图，如图 81 所示。接下来，我们可以研究锁的原因以及在这个地方使线程通信更有效的可能方法。[228]

13.3.2 平台视图

Intel VTune Profiler 的另一个非常有用的功能是平台视图（见图 82），它允许我们观察程序执行的任何给定时刻每个线程在做什么。这对于理解应用程序的行为并找到潜在的性能增长空间非常有帮助。例如，我们可以看到在从 1 秒到 3 秒的时间间隔内，只有两个线程在持续地利用相应 CPU 核心的约 100%（线程 ID 分别为 7675 和 7678）。在此期间，其他线程的 CPU 利用率是突发性的。

[228] 我不认为这将是一条容易的道路，并且不能保证您会找到使其更好的方法。

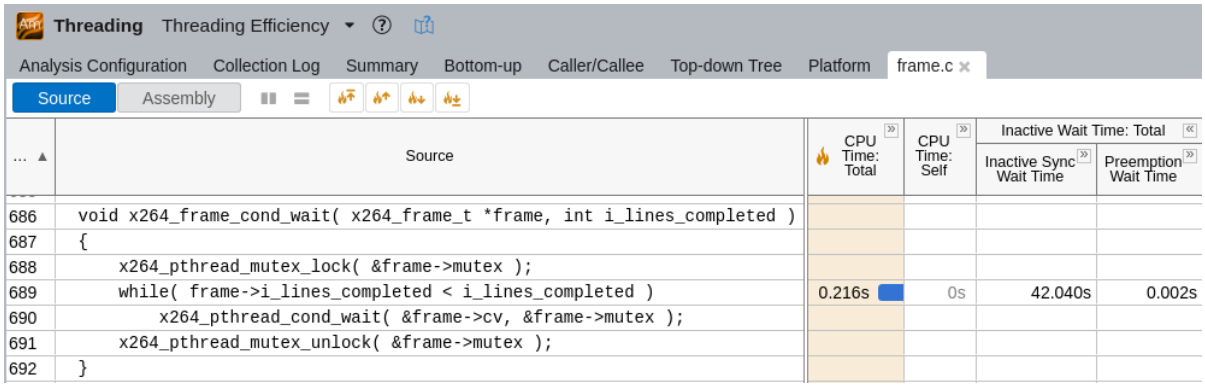


Figure 81: 显示了 Phoronix 测试套件中 x264 基准测试中 `x264_8_frame_cond_wait` 函数的源代码视图。

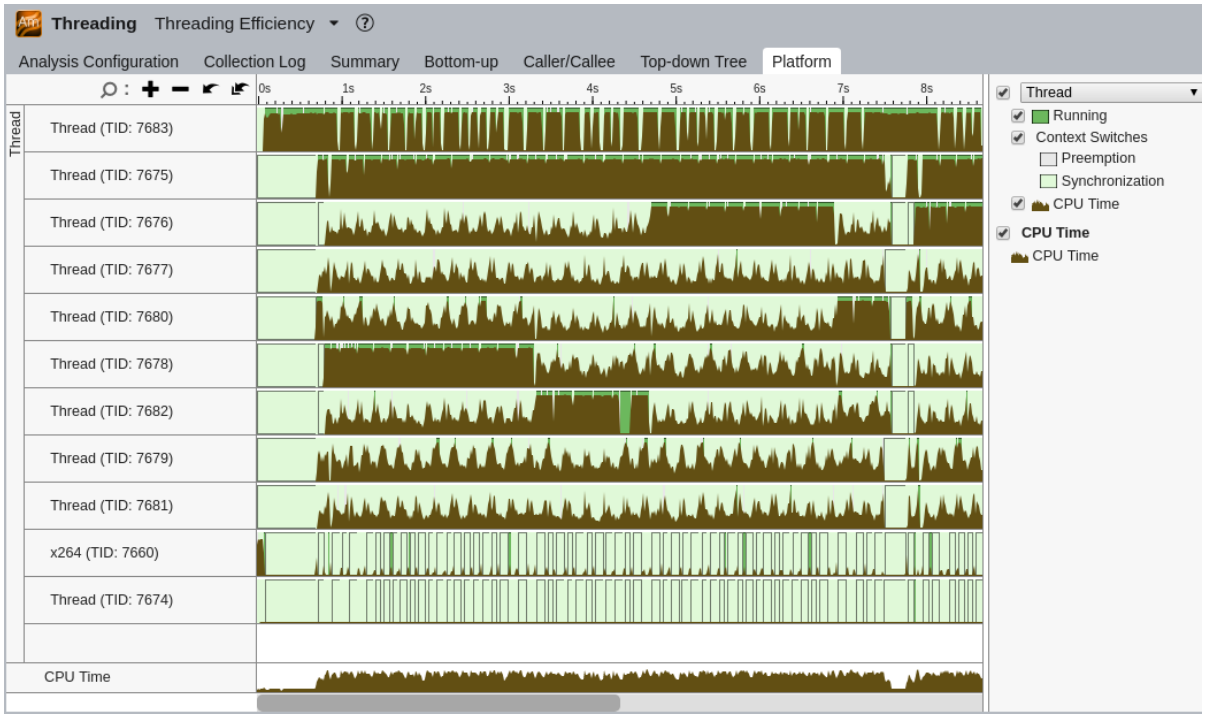


Figure 82: 显示了 Phoronix 测试套件中 x264 基准测试的 Intel VTune Profiler 平台视图。

平台视图还具有缩放和过滤功能。这使我们能够了解指定时间范围内每个线程在执行什么操作。要查看此内容，请在时间轴上选择范围，右键单击并选择“放大”和“按所选内容过滤”。Intel VTune Profiler 将显示在此时间范围内使用的函数或同步对象。

13.4 使用 Linux Perf 进行分析

Linux 的 `perf` 工具可以对应用程序可能产生的所有线程进行性能分析。它有 `-s` 选项，可以记录每个线程的事件计数。使用此选项，在报告的末尾，`perf` 列出了所有线程 ID 以及每个线程收集的样本数：

```
$ perf record -s ./x264 -o /dev/null --slow --threads 8 Bosphorus_1920x1080_120fps_420_8bit_YUV.y4m
$ perf report -n -T
...
#  PID   TID   cycles:ppp
  6966  6976  41570283106
  6966  6971  25991235047
  6966  6969  20251062678
  6966  6975  17598710694
  6966  6970  27688808973
  6966  6972  23739208014
  6966  6973  20901059568
  6966  6968  18508542853
  6966  6967     48399587
  6966  6966   2464885318
```

要为特定的软件线程过滤样本，可以使用 `--tid` 选项：

```
$ perf report -T --tid 6976 -n
# Overhead  Samples  Shared Object  Symbol
# ........  .......  .............  ..................................
     7.17%   19877       x264       get_ref_avx2
     7.06%   19078       x264       x264_8_me_search_ref
     6.34%   18367       x264       refine_subpel
     5.34%   15690       x264       x264_8_pixel_satd_8x8_internal_avx2
     4.55%   11703       x264       x264_8_pixel_avg2_w16_sse2
     3.83%   11646       x264       x264_8_pixel_avg2_w8_mmx2
```

Linux 的 `perf` 也自动提供了我们在 Section 13.2 中讨论的一些指标：

```
$ perf stat ./x264 -o /dev/null --slow --threads 8 Bosphorus_1920x1080_120fps_420_8bit_YUV.y4m
         86,720.71 msec task-clock        #    5.701 CPUs utilized
            28,386      context-switches  #    0.327 K/sec
             7,375      cpu-migrations    #    0.085 K/sec
            38,174      page-faults       #    0.440 K/sec
   299,884,445,581      cycles            #    3.458 GHz
   436,045,473,289      instructions      #    1.45  insn per cycle
    32,281,697,229      branches          #  372.249 M/sec
       971,433,345      branch-misses     #    3.01% of all branches
```

13.4.1 查找开销大的锁

要使用 Linux 的 perf 找到最有争议的同步对象，需要对调度程序上下文切换进行采样（sched:sched_switch），这是一个内核事件，因此需要 root 访问权限：

```
$ sudo perf record -s -e sched:sched_switch -g --call-graph dwarf -- ./x264 -o /dev/null --slow
    --threads 8 Bosphorus_1920x1080_120fps_420_8bit_YUV.y4m
$ sudo perf report -n --stdio -T --sort=overhead,prev_comm,prev_pid --no-call-graph -F
    overhead,sample
# Samples: 27K of event 'sched:sched_switch'
# Event count (approx.): 27327
# Overhead       Samples         prev_comm    prev_pid
# ........  ............  ................  ..........
    15.43%          4217              x264        2973
    14.71%          4019              x264        2972
    13.35%          3647              x264        2976
    11.37%          3107              x264        2975
    10.67%          2916              x264        2970
    10.41%          2844              x264        2971
     9.69%          2649              x264        2974
     6.87%          1876              x264        2969
     4.10%          1120              x264        2967
     2.66%           727              x264        2968
     0.75%           205              x264        2977
```

上面的输出显示了哪些线程最频繁地被切换出执行。请注意，我们还收集了调用堆栈（--call-graph dwarf，见 Section 5.5.3），因为我们需要用它来分析导致昂贵同步事件的路径：

```
$ sudo perf report -n --stdio -T --sort=overhead,symbol -F overhead,sample -G
# Overhead       Samples  Symbol
# ........  ............  ........................................
   100.00%         27327  [k] __sched_text_start
   |
   |--95.25%--0xffffffffffffffff
   |  |
   |  |--86.23%--x264_8_macroblock_analyse
   |  |  |
   |  |   --84.50%--mb_analyse_init (inlined)
   |  |     |
   |  |      --84.39%--x264_8_frame_cond_wait
   |  |        |
   |  |         --84.11%--__pthread_cond_wait (inlined)
   |  |                   __pthread_cond_wait_common (inlined)
   |  |                   |
   |  |                    --83.88%--futex_wait_cancelable (inlined)
   |  |                              entry_SYSCALL_64
   |  |                              do_syscall_64
   |  |                              __x64_sys_futex
```

```
  |  |                          do_futex
  |  |                          futex_wait
  |  |                          futex_wait_queue_me
  |  |                          schedule
  |  |

__sched_text_start
  ...
```

上面的列表显示了导致等待条件变量（__pthread_cond_wait）和后续上下文切换的最频繁路径。这条路径是 x264_8_macroblock_analyse -> mb_analyse_init -> x264_8_frame_cond_wait。从这个输出中，我们可以得知 84% 的上下文切换都是由线程在 x264_8_frame_cond_wait 中等待条件变量引起的。

13.5 使用 Coz 进行分析

在 Section 13.1 中，我们定义了识别影响多线程程序整体性能的代码部分的挑战。由于各种原因，优化多线程程序的一部分并不总是能带来明显效果。Coz: https://github.com/plasma-umass/coz[229] 是一种新型的性能分析器，解决了这个问题，填补了传统软件性能分析器留下的空白。它使用一种称为“因果性能分析”的新技术，通过在应用程序运行期间虚拟地加速代码段来预测某些优化的整体效果，从而进行实验。它通过插入减慢所有其他同时运行代码的暂停来实现这些“虚拟加速”。[Curtsinger & Berger, 2018]

将 Coz 分析器应用于 Phoronix 测试套件: https://www.phoronix-test-suite.com/ 中的 C-Ray: https://openbenchmarking.org/test/pts/c-ray 基准的示例如图 83 所示。根据图表，如果我们将 c-ray-mt.c 中第 540 行的性能提高 20%，Coz 预计 C-Ray 基准整体应用程序性能将相应提高约 17%。一旦我们在这条线上的改进达到 ~45%，对其应用程序的影响就会根据 Coz 的估计开始趋于平缓。有关此示例的更多详细信息，请参阅 easyperf 博客上的文章: https://easyperf.net/blog/2020/02/26/coz-vs-sampling-profilers[230]。

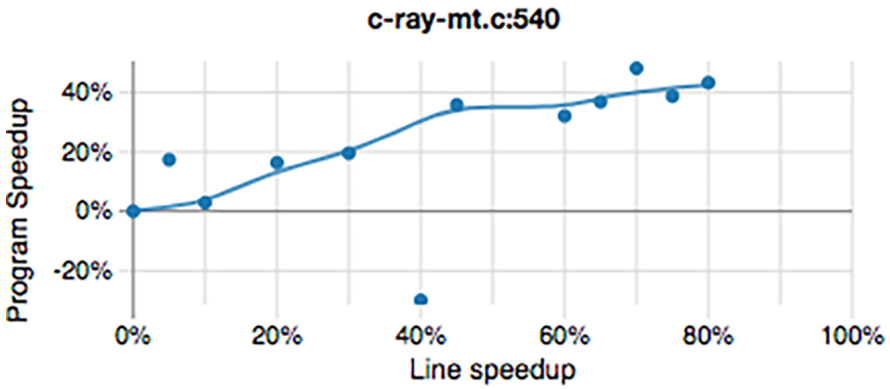


Figure 83: CozProfile 适用于 C-Ray 基准的 Coz 分析文件.

13.6 利用 eBPF 和 GAPP 进行分析

Linux 支持各种线程同步原语 - 互斥锁、信号量、条件变量等。内核通过 futex 系统调用支持这些线程原语。因此，通过追踪内核中 futex 系统调用的执行，同时从涉及的线程中收集有用的元数据，可以更轻松地识别竞争瓶颈。

[229] COZ 源代码 - https://github.com/plasma-umass/coz。
[230] 博客文章“COZ 与采样性能分析器” - https://easyperf.net/blog/2020/02/26/coz-vs-sampling-profilers。

Linux 提供了内核跟踪/分析工具，使之成为可能，其中 eBPF (Extended Berkley Packet Filter[231]) 的功能最为强大。

eBPF 基于内核中运行的沙箱虚拟机，允许在内核内安全高效地执行用户定义的程序。用户定义的程序可以用 C 语言编写，并由 BCC 编译器 (https://github.com/iovisor/bcc)[232] 编译成 BPF 字节码，以便加载到内核虚拟机中。这些 BPF 程序可以编写成在某些内核事件执行时启动，并通过各种方式将原始或处理后的数据传回用户空间。

开源社区提供了许多用于通用目的的 eBPF 程序。其中一个工具是通用自动并行分析器 (Generic Automatic Parallel Profiler) (GAPP)，它有助于跟踪多线程竞争问题。GAPP 使用 eBPF 通过对已识别的序列化瓶颈进行关键性排序来跟踪多线程应用程序的竞争开销，收集被阻塞的线程和导致阻塞的线程的堆栈轨迹。GAPP 最好的方面是它不需要代码更改、昂贵的工具化或重新编译。GAPP 分析器的创建者能够确认已知的瓶颈，并且还揭示了 Parsec 3.0 Benchmark Suite (https://parsec.cs.princeton.edu/index.htm)[233] 和一些大型开源项目中以前未报告的新瓶颈。

13.7 缓存一致性问题

13.7.1 缓存一致性协议

多处理器系统采用缓存一致性协议来确保每个包含独立缓存的独立内核共享使用内存时的数据一致性。如果没有这样的协议，假设 CPU A 和 CPU B 都将内存位置 L 读取到各自的缓存中，然后处理器 B 随后修改其缓存值 L，那么 CPU 将具有相同内存位置 L 的不一致值。缓存一致性协议确保对缓存条目的任何更新都忠实地更新在同一位置的任何其他缓存条目中。

MESI（Modified Exclusive Shared Invalid）是最著名的缓存一致性协议之一，用于支持现代 CPU 中使用的回写缓存。其缩写表示缓存行可以标记的四种状态（参见图 84）：

- **修改 (Modified)**：缓存行仅存在于当前缓存中，并且已从其在 RAM 中的值进行修改 (Cache 上的数据已经被更新过，但是还没有写到内存里)
- **独占 (Exclusive)**：缓存行仅存在于当前缓存中，并且与其在 RAM 中的值匹配 (数据只存储在一个 CPU 核心的 Cache 里，而其他 CPU 核心的 Cache 没有该数据; 如果要向独占的 Cache 写数据，就可以直接自由地写入，而不需要通知其他 CPU 核心，因为只有你这有这个数据，就不存在缓存一致性的问题)
- **共享 (Shared)**：缓存行存在于这里和其他缓存行中，并且与其在 RAM 中的值匹配 (相同的数据在多个 CPU 核心的 Cache 里都有，所以当我们要更新 Cache 里面的数据的时候，不能直接修改，而是要先向所有的其他 CPU 核心广播一个请求，要求先把其他核心的 Cache 中对应的 Cache Line 标记为「无效」状态，然后再更新当前 Cache 里面的数据)
- **无效 (Invalid)**：缓存行未使用（即不包含任何 RAM 位置）（数据已经失效了，不可以读取该状态的数据）

从内存中获取时，每个缓存行都将一个状态编码到其标签中。然后，缓存行状态会从一个状态转换到另一个状态。[234] 现实中，CPU 供应商通常会实现稍作改进的 MESI 变体。例如，英特尔使用 MESIF: https://en.wikipedia.org/wiki/MESIF_protocol，[235] 它添加了转发 (F) 状态，而 AMD 则使用 MOESI: https://en.wikipedia.org/wiki/MOESI_protocol，[236] 它添加了拥有 (O) 状态。但这些协议仍然保持了基本 MESI 协议的本质。

正如早期示例所示，缓存一致性问题会导致程序出现顺序不一致的问题。这个问题可以通过使用窥探缓存来监视所有内存事务并相互协作以保持内存一致性来缓解。不幸的是，这也伴随着成本，因为一个处理器的修改会使另一个处理器缓存中的对应缓存行失效。这会导致内存停顿并浪费系统带宽。与只能为应用程序性能设置上限的序列化和锁定问题不同，一致性问题会导致由 USL 在 Section 13.1 中描述的逆行效应。两种广为人知的缓存一致性问题是“真共享”和“伪共享”，我们将在下面进一步探讨。

[231] eBPF 文档 - https://prototype-kernel.readthedocs.io/en/latest/bpf/

[232] BCC 编译器 - https://github.com/iovisor/bcc

[233] Parsec 3.0 基准测试套件 - https://parsec.cs.princeton.edu/index.htm

[234] 读者可以在此处观看和测试动画 MESI 协议：https://www.scss.tcd.ie/Jeremy.Jones/vivio/caches/MESI.htm.

[235] MESIF - https://en.wikipedia.org/wiki/MESIF_protocol

[236] MOESI - https://en.wikipedia.org/wiki/MOESI_protocol

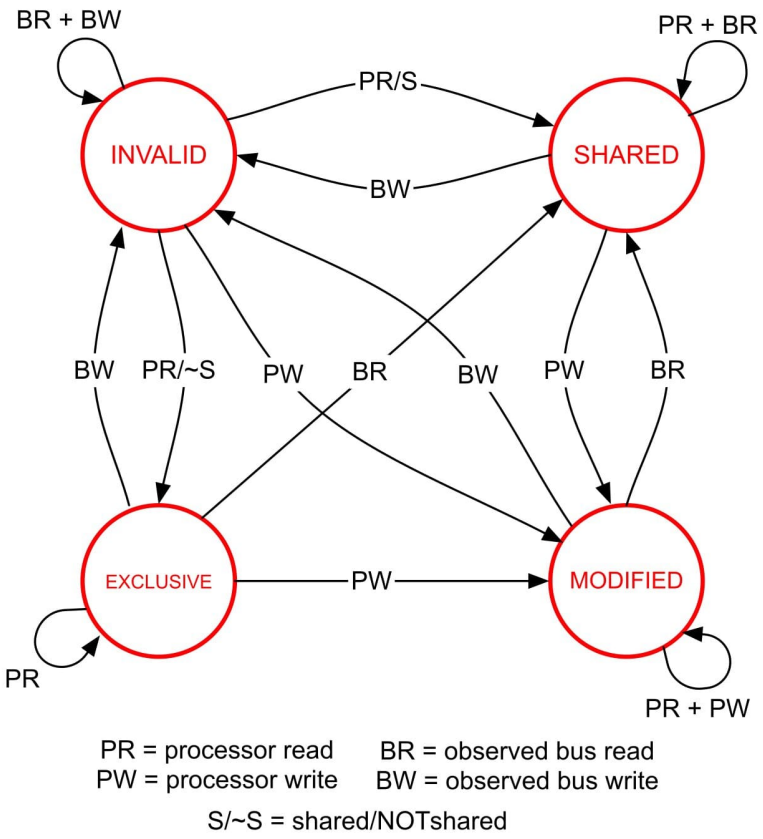


Figure 84: MESI 状态图. © Image by University of Washington via courses.cs.washington.edu.

13.7.2 真共享

真共享指的是两个不同的处理器访问同一个变量（请参见 Listing 13.7.2）。

代码清单: 真正的共享示例。

```
unsigned int sum;
{ // parallel section
  for (int i = 0; i < N; i++)
    sum += a[i]; // sum is shared between all threads
}
```

真实共享意味着存在数据竞争，这很难被检测到。幸运的是，有一些工具可以帮助识别这类问题。Clang 的 Thread sanitizer: https://clang.llvm.org/docs/ThreadSanitizer.html[237] 和 helgrind: https://www.valgrind.org/docs/manual/hg-manual.html[238] 就是其中的一些工具。为了防止 Listing 13.7.2 中的数据竞争，应该将 `sum` 变量声明为 `std::atomic<unsigned int> sum`。

当发生真实共享时，使用 C++ 原子类型可以帮助解决数据竞争问题。然而，它实际上序列化了对原子变量的访问，这可能会降低性能。解决真实共享问题的另一种方法是使用线程局部存储 (TLS)。TLS 是一种方法，允许给定多线程进程中的每个线程分配内存来存储线程特定的数据。通过这样做，线程修改自己的本地副本，而不是竞争全局可用的内存位置。可以使用 TLS 类说明符 (`thread_local unsigned int sum`，自 C++11 起) 声明 `sum` 来修复 Listing 13.7.2 中的示例。然后，主线程应该合并每个工作线程所有本地副本的结果。

13.7.3 伪共享

伪共享[239] 发生在两个不同的处理器修改位于同一缓存行上的不同变量时（参见 Listing 239）。图 85 展示了伪共享问题。

代码清单: 伪共享示例。

```
struct S {
  int sumA; // sumA and sumB are likely to
  int sumB; // reside in the same cache line
};
S s;

{ // section executed by thread A
  for (int i = 0; i < N; i++)
    s.sumA += a[i];
}

{ // section executed by thread B
  for (int i = 0; i < N; i++)
    s.sumB += b[i];
}
```

[237] Clang 的线程 Sanitizer 工具:https://clang.llvm.org/docs/ThreadSanitizer.html。

[238] Helgrind，一个线程错误检测工具:https://www.valgrind.org/docs/manual/hg-manual.html。

[239] 值得注意的是，错误共享不仅在 C/C++/Ada 等低级语言中可以观察到，在 Java/C# 等高级语言中也可以观察到。

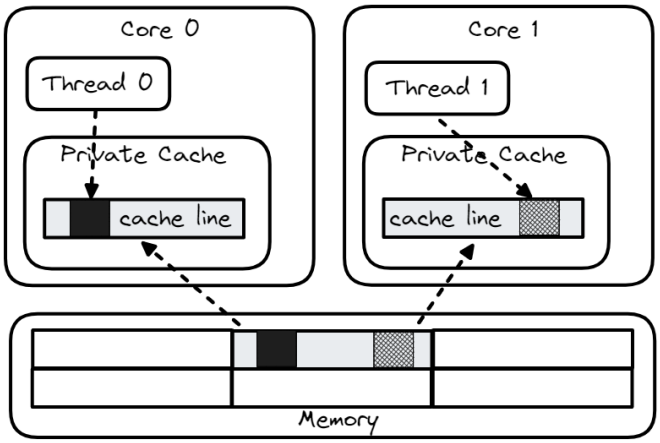


Figure 85: False 共享: 两个线程访问同一个缓存行。© Image by Intel Developer Zone via software.intel.com.

伪共享是多线程应用程序性能问题的常见来源。因此，现代分析工具内置了检测此类案例的支持。TMA 将经历真/伪共享的应用程序描述为内存瓶颈 (Memory Bound)。通常，在这种情况下，您会看到 竞争访问: https://software.intel.com/en-us/vtune-help-contested-accesses[240] 指标的高值。

使用 Intel VTune Profiler 时，用户需要两种类型的分析来查找和消除伪共享问题。首先，运行 微架构探索: https://software.intel.com/en-us/vtune-help-general-exploration-analysis[241] 分析，该分析实施 TMA 方法来检测应用程序中是否存在伪共享。正如之前提到的，竞争访问指标的高值促使我们深入挖掘并运行启用了“分析动态内存对象”选项的 内存访问: https://software.intel.com/en-us/vtune-help-memory-access-analysis 分析。此分析有助于找出导致竞争问题的对数据结构的访问。通常，这些内存访问具有高延迟，分析会揭示这一点。有关使用 Intel VTune Profiler 修复伪共享问题的示例，请参见 英特尔开发者社区: https://software.intel.com/en-us/vtune-cookbook-false-sharing[242]。

Linux `perf` 也支持查找伪共享。与 Intel VTune Profiler 一样，首先运行 TMA（请参见 Section 6.1.1）以找出程序是否经历假/真共享问题。如果是这种情况，请使用 `perf c2c` 工具检测具有高缓存一致性成本的内存访问。`perf c2c` 匹配不同线程的存储/加载地址，并查看是否命中了修改后的缓存行。读者可以在专门的 博客文章: https://joemario.github.io/blog/2016/09/01/c2c-blog/[243] 中找到该过程及其如何使用工具的详细解释。

可以通过对齐/填充内存对象来消除伪共享。Section 13.7.2 中的示例可以通过确保 `sumA` 和 `sumB` 不共享同一缓存行来修复（请参阅 Section 8.1.4 中的详细信息）。

从一般的性能角度来看，最重要的考虑因素是可能状态转换的成本。在所有缓存状态中，唯一不涉及昂贵的跨缓存子系统通信和 CPU 读/写操作期间的数据传输的是修改 (M) 和独占 (E) 状态。因此，缓存行保持“M”或“E”状态的时间越长（即跨缓存的数据共享越少），多线程应用程序产生的一致性成本就越低。有关如何利用此属性的示例，请参见 Nitsan Wakart 的博客文章“深入了解缓存一致性: http://psy-lob-saw.blogspot.com/2013/09/diving-deeper-into-cache-coherency.html”[244]。

问题和练习

1. 完成 `perf-ninja::false_sharing` 实验练习。
2. 运行你日常使用的应用程序。它是否多线程？如果不是，请选用一些多线程基准测试。计算并行效率指标，并运行扩展性研究。观察时间线图，是否存在调度问题？识别热点锁以及导致这些锁的代码路径。你能改进锁定

[240] 竞争访问 - https://software.intel.com/en-us/vtune-help-contested-accesses.
[241] Vtune 一般探索分析 - https://software.intel.com/en-us/vtune-help-general-exploration-analysis.
[242] Vtune 食谱：伪共享 - https://software.intel.com/en-us/vtune-cookbook-false-sharing.
[243] 关于 `perf c2c` 的文章 - https://joemario.github.io/blog/2016/09/01/c2c-blog/.
[244] 博客文章“深入缓存一致性”- http://psy-lob-saw.blogspot.com/2013/09/diving-deeper-into-cache-coherency.html

吗？检查应用程序性能是否受到真/伪共享的影响。

3. 奖励问题：多线程应用与多进程应用相比有哪些优势？

AI 回答：

1. **完成 `perf-ninja::false_sharing` 实验练习：**
   - 这个实验练习通常涉及使用 Linux 的`perf`工具来检测程序中的伪共享问题。伪共享发生在多个线程频繁访问位于同一缓存行上的不同变量时。要完成这个练习，你需要编写或获取一个多线程程序，使用`perf`工具运行`perf-ninja`命令，并指定`false_sharing`作为检测目标。然后，分析输出结果，识别可能的伪共享问题，并尝试通过调整数据结构的布局或使用线程局部存储（TLS）来解决这些问题。
2. **运行日常使用的应用程序并分析：**
   - 首先，确定你的应用程序是否是多线程的。可以通过查看应用程序的文档或源代码来确认。如果应用程序不是多线程的，可以选择一些多线程基准测试，如`h264dec`或`x264`编码测试。
   - 使用性能分析工具（如 Intel VTune Profiler 或 Linux Perf）来计算并行效率指标，这通常涉及到测量 CPU 利用率、线程数、等待时间和自旋时间。
   - 运行扩展性研究，观察随着线程数量的增加，应用程序性能如何变化。这可以通过增加线程数并重新运行性能测试来完成。
   - 分析时间线图，检查是否存在调度问题，如线程频繁上下文切换或等待时间异常高。
   - 识别热点锁，即那些导致线程等待的锁。这可以通过分析工具中的锁分析功能来完成。
   - 检查代码路径，找出导致热点锁的原因，并考虑是否可以通过改进锁的粒度、使用无锁数据结构或其他同步机制来优化。
   - 检查应用程序性能是否受到真/伪共享的影响。这可以通过分析工具中的内存访问模式来识别。
3. **奖励问题：多线程应用与多进程应用的优势：**
   - **资源共享**：多线程应用可以在同一个进程空间内共享内存和资源，这比多进程应用更高效，因为进程间通信（IPC）通常比线程间通信更复杂和耗时。
   - **上下文切换开销**：线程的上下文切换通常比进程的上下文切换要快，因为线程共享相同的地址空间和资源，而进程则需要独立的地址空间。
   - **响应时间**：多线程应用可以更快地响应用户输入或事件，因为线程可以在不同的 CPU 核心上并行执行，从而提高整体的响应速度。
   - **简化设计**：在某些情况下，多线程模型可以简化程序设计，因为它允许开发者在单个程序中组织并行任务，而不是分散到多个进程中。
   - **资源利用率**：多线程应用可以更好地利用多核处理器的资源，因为线程可以在多个核心上并行执行，而多进程应用可能需要额外的协调来实现这一点。

请注意，虽然多线程应用有这些优势，但它们也带来了额外的复杂性，如同步和并发控制问题。开发者需要仔细设计和测试多线程程序，以确保它们在并发环境下的正确性和性能。

章节总结

- 没有利用现代多核 CPU 的应用程序正在落后于竞争对手。为了应用程序未来的成功，让软件能够良好地随着 CPU 内核数量的增长而扩展是非常重要的。
- 当处理单线程应用程序时，优化程序的一部分通常会对性能产生积极影响。然而，对于多线程应用程序来说，情况不一定如此。这种效应被称为阿姆达尔定律，它规定并行程序的加速受其串行部分的限制。
- 如通用可扩展性定律所解释，线程通信可能导致负面加速。这给多线程程序的调优带来了额外的挑战。优化多线程应用程序的性能还涉及检测和减轻竞争和一致性的影响。
- Intel VTune Profiler 是分析多线程应用程序的常用工具。但是近年来，其他工具也出现了，它们拥有独特的功能集，例如 Coz 和 GAPP。